

Galaxy-X-os — One-Page Technical Report

SCALE x ODYSSEY | Deep-learning classification of raw astronomical images into 5 celestial categories.

GitHub: <https://github.com/Srujan0798/Galaxy-X-os>

Dataset Sources

- **Spiral Galaxy + Elliptical Galaxy** — real survey imagery from **Galaxy10 DECaLS** (via `astroNN`; real SDSS/DECaLS images), mapped from Galaxy10's 10 morphology classes into our two galaxy-morphology classes.
- **Nebula + Star Cluster + Planetary Object** — deep-space image datasets pulled via a Kaggle token-authenticated download (`fedesoriano/deep-space-images` for Nebula & Star Cluster; `brsdincer/planetary-solar-system-objects` for Planetary), with a clearly-labelled **procedural fallback** for any class whose dataset was unavailable at run time.
- **5 classes:** Spiral Galaxy, Elliptical Galaxy, Nebula, Star Cluster, Planetary Object.
- **Split:** stratified **disjoint** train / val / test. Verified by MD5 hash that no image appears in more than one split (zero train/test leakage).
- **Source of truth:** the authoritative per-class real-vs-fallback record is `data/processed/DATA_MANIFEST.json`, generated by `src/prepare_data.py`. We do **not** claim all data is real telescope imagery — the two galaxy classes are real survey images; the other three come from the Kaggle deep-space sets or a labelled procedural fallback.

Model Architecture

- **Backbone:** EfficientNet-B3 (ImageNet pretrained, transfer learning) — ~11.6M parameters.
- **Head:** Replaced classifier → 5-class output.
- **Training strategy:** Progressive unfreezing (backbone frozen first 3 epochs), mixed-precision (`torch.amp`), AdamW (lr $3e-4$), batch size 32, OneCycleLR scheduler.
- **Augmentations:** Albumentations astro-specific pipeline (cosmic-ray simulation, vignetting, Poisson noise, flips/rotations, label smoothing 0.1).
- **Explainability:** Grad-CAM heatmaps over predictions.

Final Test Metrics

Evaluated on the held-out **disjoint** test set (250 images, 50 per class — none seen in training). Macro-averaged.

Metric	Standard	+ TTA (6× aug)
Accuracy	95.6%	96.4%
Precision (macro)	0.958	—
Recall (macro)	0.956	—
F1 (macro)	0.956	0.964

Per-class F1 (Standard):

Class	F1
Spiral Galaxy	0.884
Elliptical Galaxy	0.895
Nebula	1.000
Star Cluster	1.000
Planetary Object	1.000

*The perfect 1.000 F1 on Nebula / Star Cluster / Planetary reflects that these classes are visually very distinct (and, where the procedural fallback was used, procedurally clean). The two **real** galaxy-morphology classes — Spiral vs Elliptical — show the realistic ~0.88–0.90 F1 that captures genuine morphological ambiguity, which is the residual error. TTA adds +0.8% accuracy.*

See `data/processed/DATA_MANIFEST.json` for the honest per-class source breakdown (which classes came from real Galaxy10 / Kaggle vs procedural fallback).

Confusion Matrix

See `results/confusion_matrix.png`. Additional plots: `results/per_class_metrics.png`, `results/training_curves.png`, `results/confidence_distribution.png`.

Inference Time

- **Measured ~72 ms per image (median; 118 ms mean) on Apple MPS** (EfficientNet-B3, batch=1, after warmup) — well under the 5 s requirement.
- Faster on a CUDA GPU; code supports `torch.autocast("cuda")` for GPU deployment.
- Streamlit demo runs in real time on consumer hardware.

Setup Instructions

```
pip install -r requirements.txt
```

```
python src/prepare_data.py      # download + build dataset (writes DATA_MANIFEST.json)
python src/train.py             # train (full EfficientNet-B3 fine-tune)
python src/evaluate.py          # evaluate (standard + TTA)
python src/gradcam.py           # Grad-CAM report
streamlit run app/app.py        # web demo
```

One-click alternative: open `notebooks/Galaxy_X_Colab.ipynb` in Google Colab to run the full pipeline end-to-end on a free GPU.

Training Script

- **Full pipeline:** `src/train.py` — progressive unfreezing (backbone frozen epochs 1-3, full fine-tune from epoch 4) + OneCycleLR, AdamW ($1r\ 3e-4$), batch 32, mixed precision (AMP), class weights, label smoothing.
- **Config:** `configs/config.yaml`. Checkpoint: `checkpoints/best_model.pth`.

Training note: the reported model was trained to **95.6% / 96.4%** on a **CUDA GPU (Google Colab)** over a 50-epoch schedule via `notebooks/Galaxy_X_Colab.ipynb`. Inference runs comfortably on consumer hardware — measured ~72 ms/image on an Apple-MPS MacBook.